



Algorithms and Data Structures

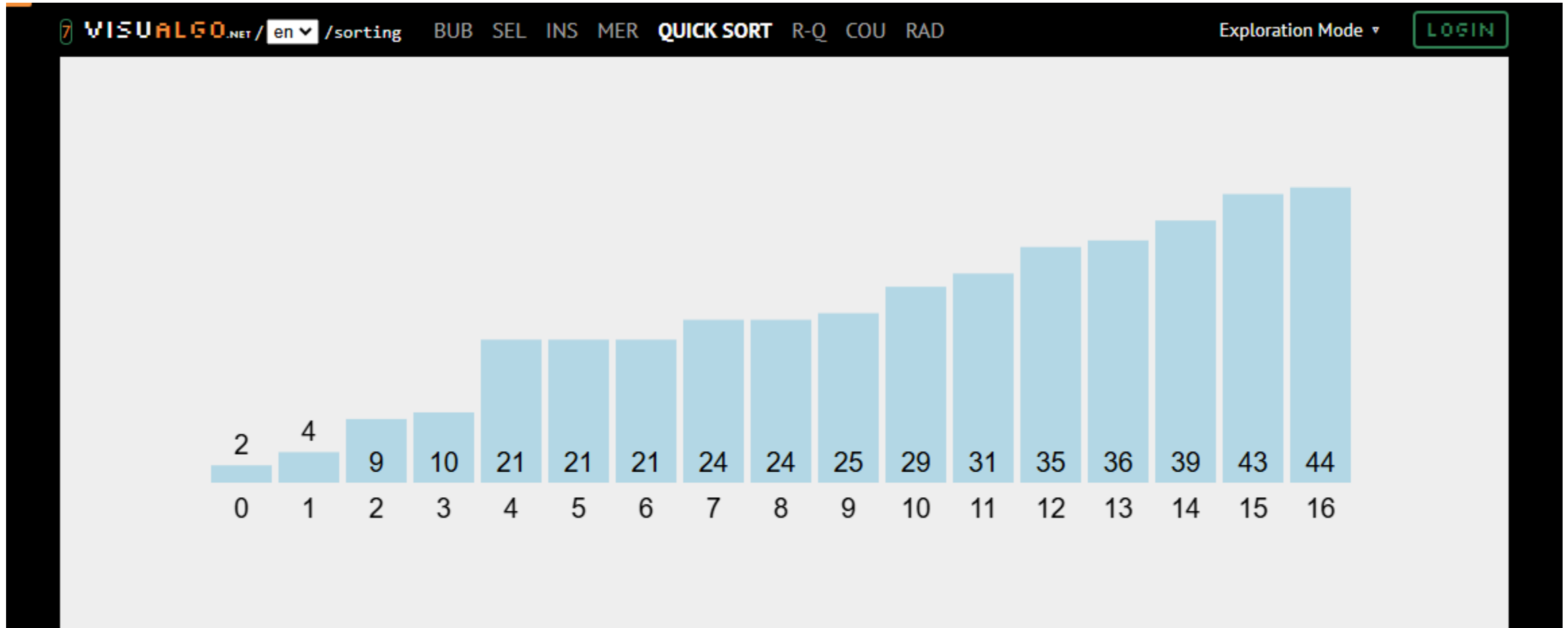


UNIVERSITY OF
MARYLAND

QuickSort

Slide Credit: These slides are adapted from materials by Prof. Manocha and Prof. Ayhan, with modifications and additions by Sujeong Kim.

QuickSort



<https://visualgo.net/en/sorting>

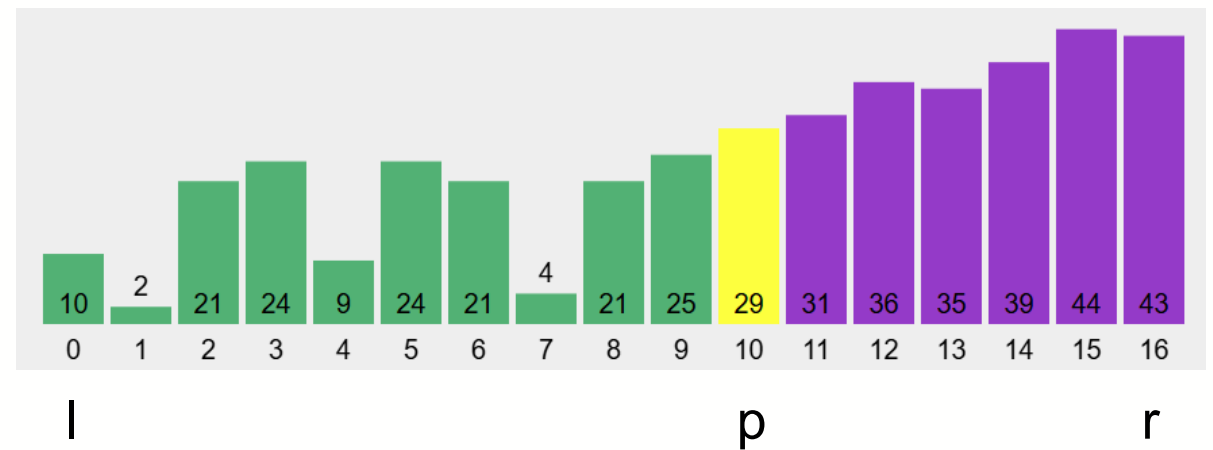
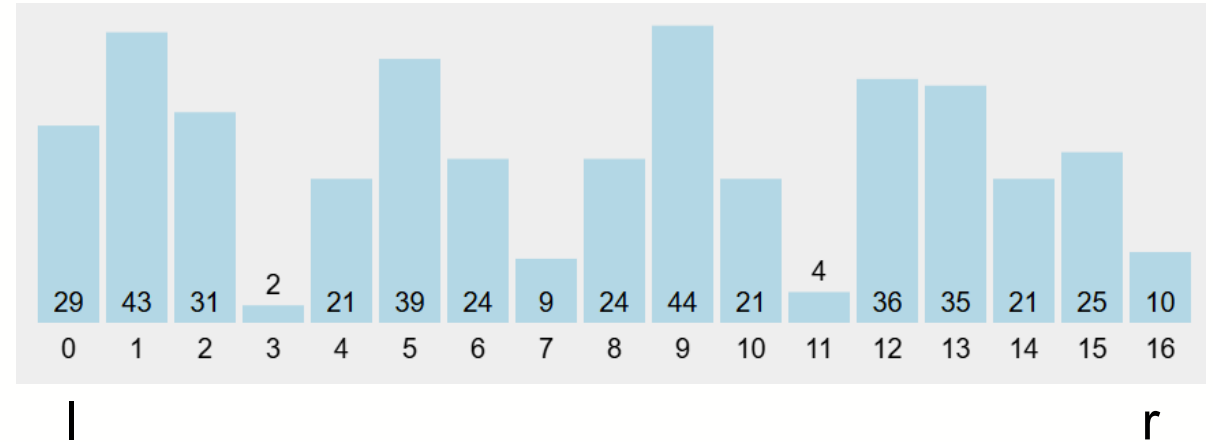
QuickSort

- **Divide:** $A[l...r]$ is partitioned (rearranged) into two nonempty subarrays $A[l...p]$ and $A[p+1...r]$ s.t. each element of $A[l...p]$ is less than or equal to each element of $A[p+1...r]$. Index p is computed here.
- **Conquer:** two subarrays are sorted by recursive calls to quicksort.
- **Combine:** no work needed since the subarrays are sorted in place already.

Quicksort (A, l, r)

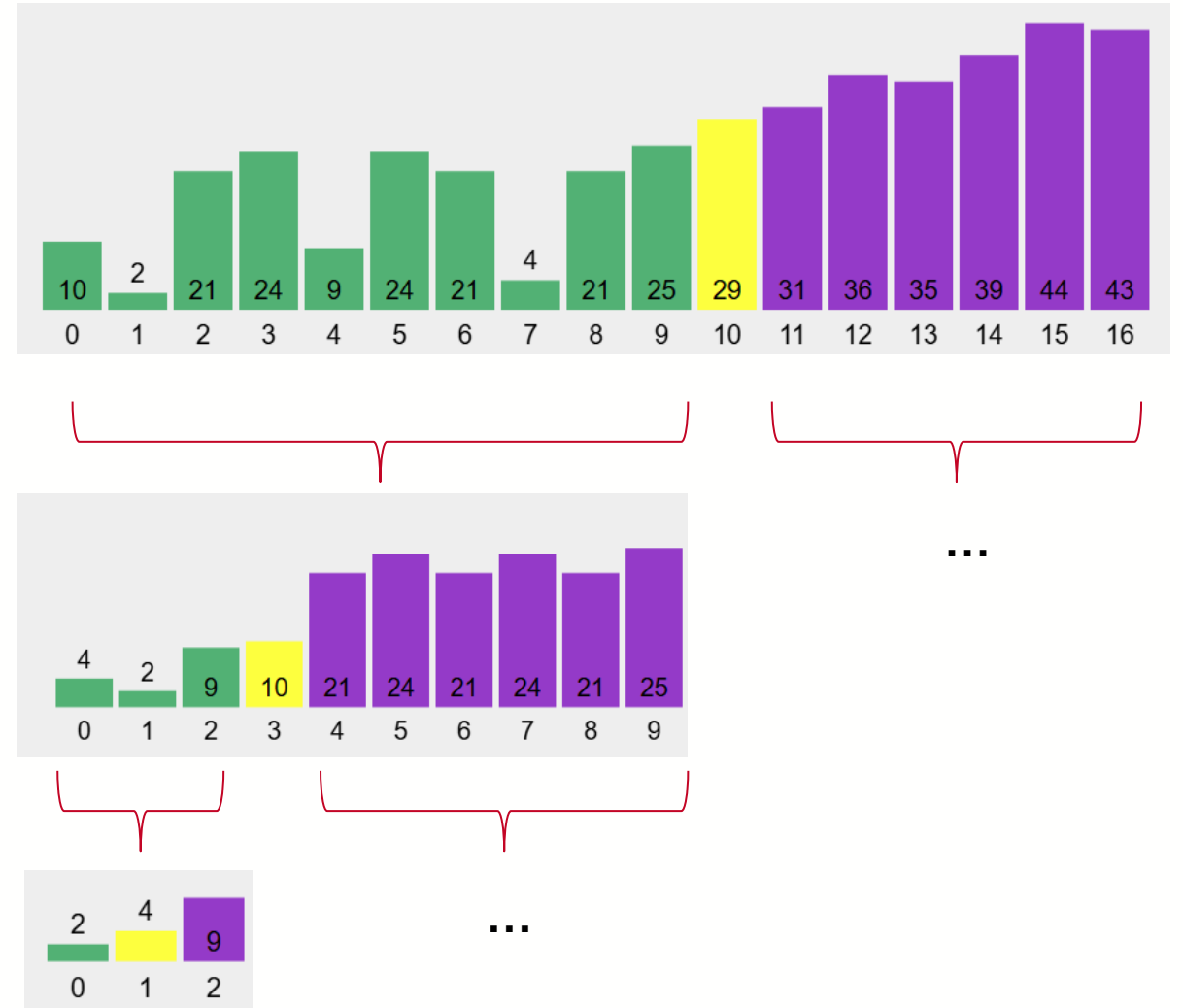
1. if $l < r$
2. then $p \leftarrow \text{Partition}(A, l, r)$
3. $\text{Quicksort}(A, l, p)$
4. $\text{Quicksort}(A, p+1, r)$

* In place, not stable



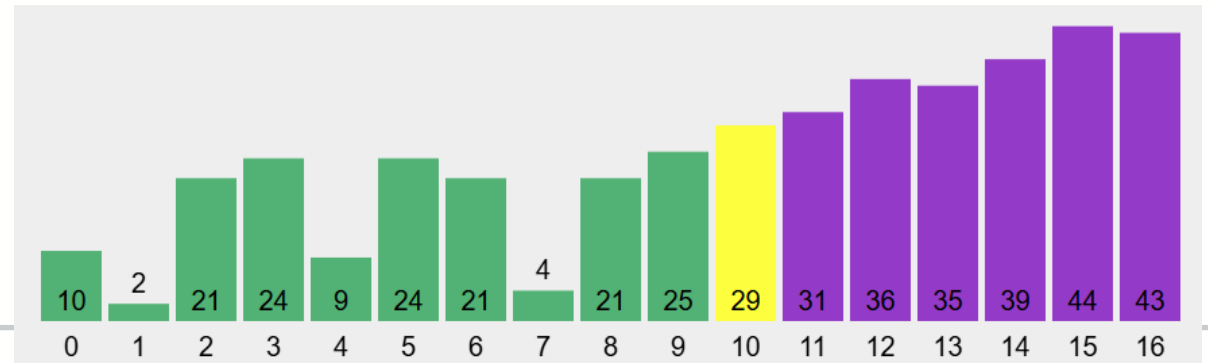
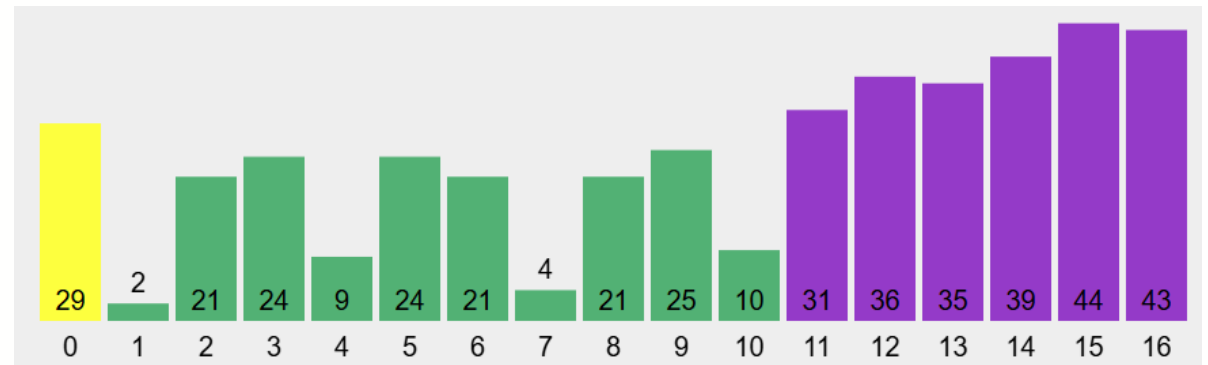
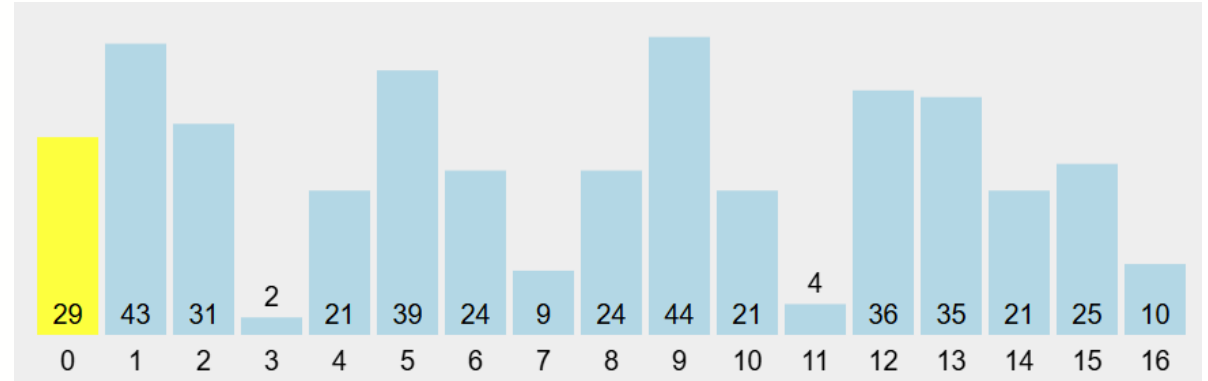
Quick Sort - Recursion

- Recursively apply the partitioning process to the subarrays of elements smaller than the pivot and larger than the pivot
- Base case: subarrays of size one or zero



Quick Sort - Partitioning

- We select a pivot element and arrange the array such that all elements less than the pivot come before the pivot, and all elements greater than the pivot come after the pivot
- You can think of this process as finding the sorted position for the pivot



Partition(A, l, r)

1. $x \leftarrow A[l]$
2. $i \leftarrow l$
3. $j \leftarrow r + 1$
4. while TRUE do
 5. repeat $j \leftarrow j - 1$
 6. until $A[j] \leq x$
 7. repeat $i \leftarrow i + 1$
 8. until $A[i] \geq x$
 9. if $i < j$ then
 10. exchange $A[i] \leftrightarrow A[j]$
 11. else
 12. exchange $A[l] \leftrightarrow A[j]$
 13. return j

Partitioning

- We select a pivot, i.e. an element in the array
- We search from the right end and find the first element that is less than or equal to the pivot
- We search from the left end and find the first element that is greater than or equal to the pivot
- We swap these two elements
- We repeat these steps until we are done

Partitioning Example (1/3)

pivot

4	3	6	9	2	4	3	1	2	1	8	9	3	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

search j

4	3	6	9	2	4	3	1	2	1	8	9	3	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

search i

i →

← j

swap $A[i]$, $A[j]$

4	3	3	9	2	4	3	1	2	1	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

i

j

Partitioning Example (2/3)

4	3	3	9	2	4	3	1	2	1	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

i

j

search j

4	3	3	9	2	4	3	1	2	1	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

search i

i →

← j

swap A[i], A[j]

4	3	3	1	2	4	3	1	2	9	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

i

j

Partitioning Example (3/3)

search j

4	3	3	1	2	4	3	1	2	9	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

search i

i → ← j

swap A[i], A[j]

4	3	3	1	2	2	3	1	4	9	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

i j

search j

4	3	3	1	2	2	3	1	4	9	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

← j

search i

i →

i > j

swap
A[l] (pivot),
A[j]

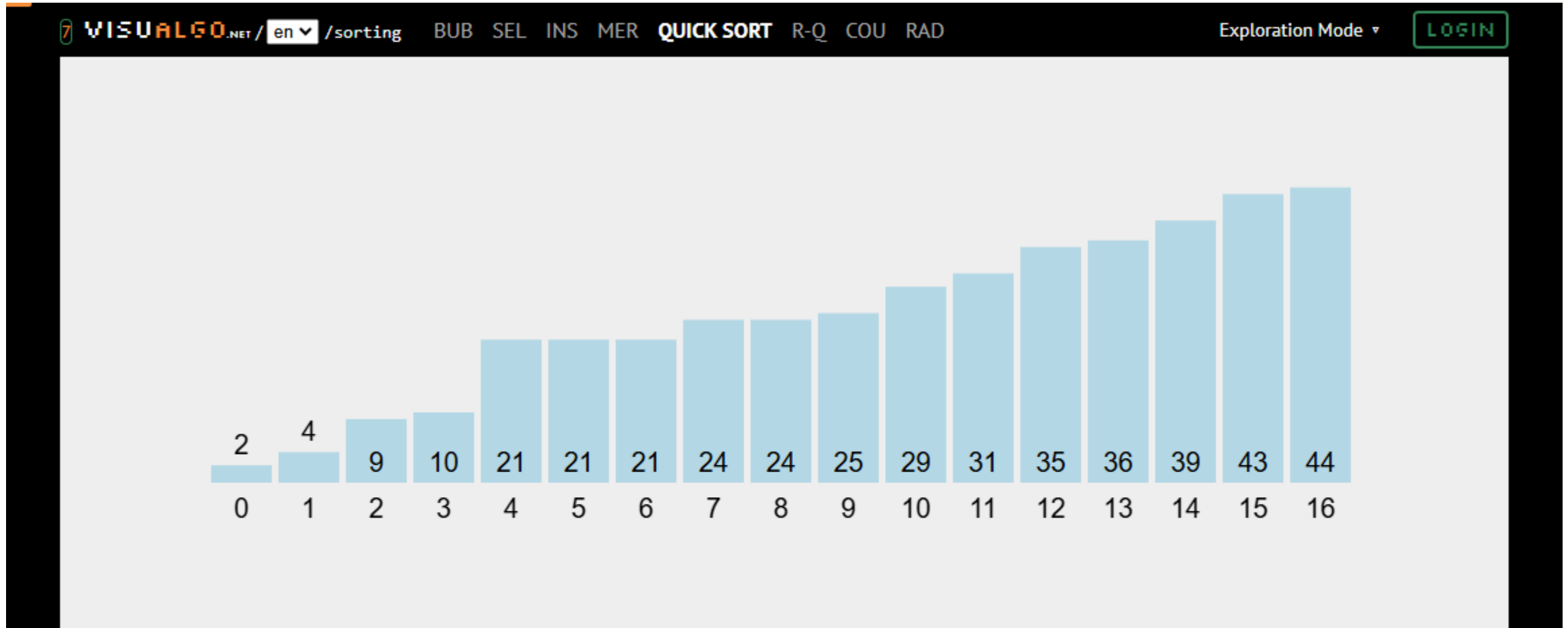
4	3	3	1	2	2	3	1	4	9	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

l

j

1	3	3	1	2	2	3	4	4	9	8	9	6	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

QuickSort



<https://visualgo.net/en/sorting>

Algorithm Analysis

- General Recurrence Relation

$$T(n) = T(an) + T((1 - a)n) + \Theta(n),$$

where a is a fraction (e.g., $a = 0.5$ for balanced splits).

Partitioning takes $\Theta(n)$.

- Base case:

$$T(1) = 1$$

there is only one element in the array (or subarray).

The algorithm returns the array.

Average-Case Behavior

- For example, when the partitioning algorithm always produces a 7-to-3 proportional split (i.e., $a = 0.7$):

$$T(n) = T\left(\frac{7}{10}n\right) + T\left(\frac{3}{10}n\right) + n$$

Solve the recurrence by visualizing recursion tree, each level has a cost of n with the height of $\lg n$. So, we get $T(n) = \Theta(n \log n)$ when the split has constant proportionality.

- For a split of proportionality a , where $0 \leq a \leq \frac{1}{2}$, the minimum depth of the tree is $-\frac{\lg n}{\lg a}$ ($= -\log_a n$) and the maximum depth is $-\frac{\lg n}{\lg(1-a)}$ ($= -\log_{1-a} n$).

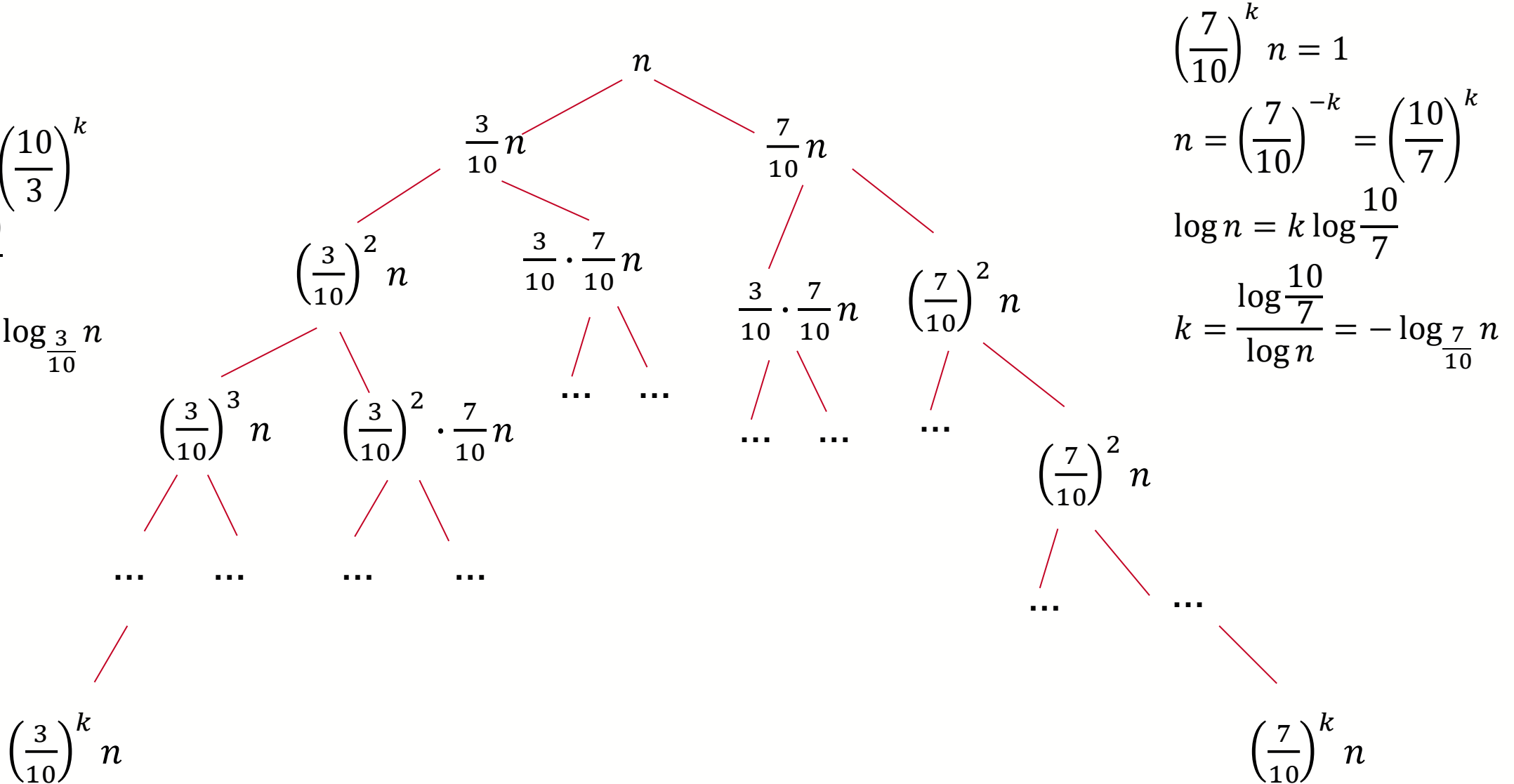
Recursion Tree - Average-Case Behavior

$$\left(\frac{3}{10}\right)^k n = 1$$

$$n = \left(\frac{3}{10}\right)^{-k} = \left(\frac{10}{3}\right)^k$$

$$\log n = k \log \frac{10}{3}$$

$$k = \frac{\log n}{\log \frac{10}{3}} = -\log_{\frac{3}{10}} n$$



Analysis for Best Case Partition

- The running time of quicksort depends on whether the partitioning is balanced or not.
- In the best case, each partition operation divides the array in half (exactly)
 - i.e., pivot is the median
- When the partitioning procedure produces two regions of size $\frac{n}{2}$, we get a balanced partition:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

$$\text{So, } T(n) = \Theta(n \log n)$$

- Can it perform better than $O(n \log n)$ on any input?

Best Case Partitioning

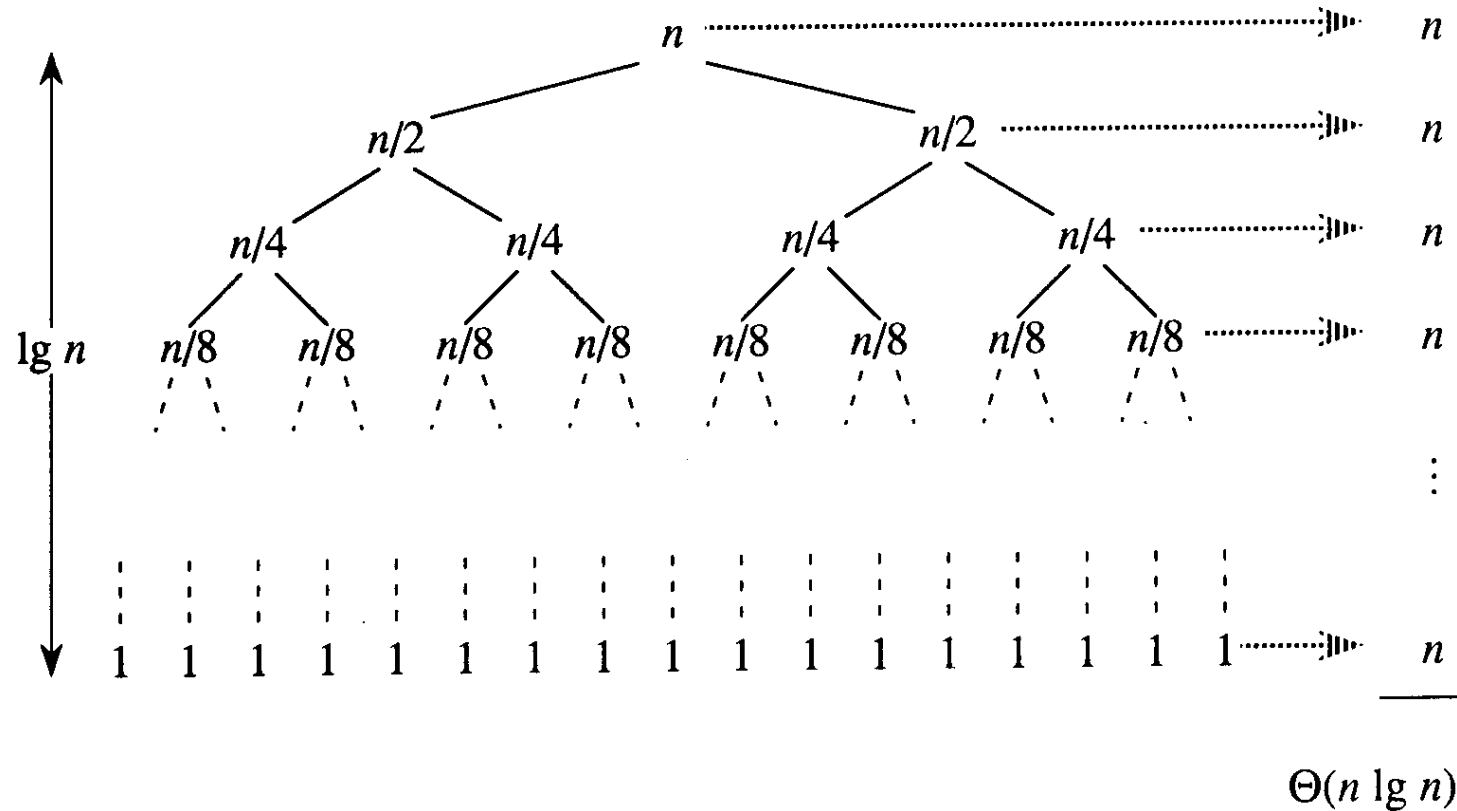
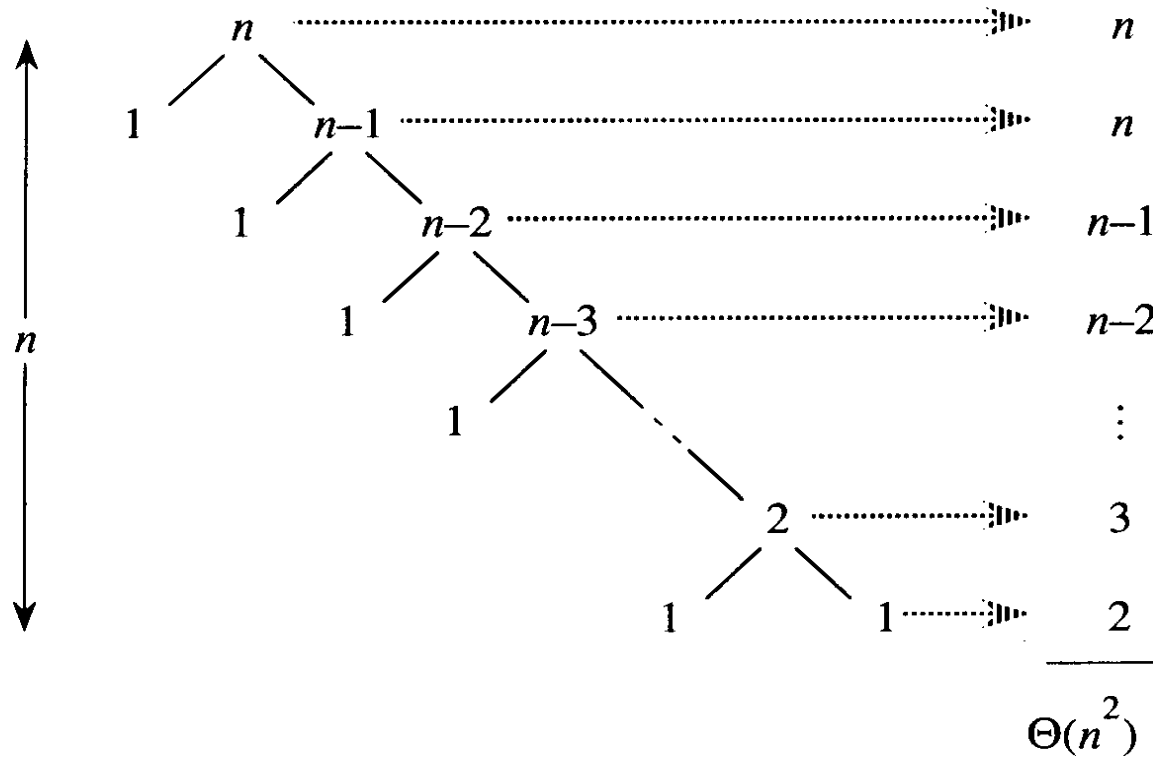


Figure 8.3 A recursion tree for QUICKSORT in which PARTITION always balances the two sides of the partition equally (the best case). The resulting running time is $\Theta(n \lg n)$.

Worst Case Partitioning



When does it happen?

Figure 8.2 A recursion tree for QUICKSORT in which the PARTITION procedure always puts only a single element on one side of the partition (the worst case). The resulting running time is $\Theta(n^2)$.

Worst Case Partitioning

- Worst-Case Performance (unbalanced):

$$\begin{aligned}T(n) &= T(1) + T(n-1) + \Theta(n) \Leftarrow \text{partitioning takes } \Theta(n) \\&= \sum_{k=1 \text{ to } n} \Theta(k) \Leftarrow T(1) \text{ takes } \Theta(1) \text{ time \& reiterate} \\&= \Theta\left(\sum_{k=1 \text{ to } n} k\right) \\&= \Theta(n^2)\end{aligned}$$

* This occurs when the input is completely sorted.

Randomized Quicksort

- An algorithm is randomized if its behavior is determined not only by the input but also on random values generated during its execution.
- Why randomize Quicksort?
 - The running time of Quicksort depends heavily on how the pivot divides the array
 - In deterministic Quicksort, certain input orders (e.g., already sorted or reverse-sorted arrays) can consistently trigger worst-case behavior

Randomized Quicksort

- Two possible versions of randomized Quicksort
 - Impose a distribution on input: Shuffle the array randomly
 - Random pivot selection: Exchange pivot with an element chosen at random from $A[l\dots r]$ in Partition. This ensures that the pivot element is equally likely to be any of input elements.
- Key Properties of Randomized Quicksort
 - No input consistently triggers worst case
 - Randomization does not change the worst-case time complexity $O(n^2)$, but it makes such cases extremely rare
 - Expected running time is always $O(n \log n)$
 - The running time becomes independent of the original input ordering

Randomized-Quicksort (A, l, r)

1. if $l < r$
2. then $p \leftarrow \text{Randomized-Partition}(A, l, r)$
3. Randomized-Quicksort(A, l, p)
4. Randomized-Quicksort(A, p+1, r)

(recall) Quicksort (A, l, r)

1. if $l < r$
2. then $p \leftarrow \text{Partition}(A, l, r)$
3. Quicksort(A, l, p)
4. Quicksort(A, p+1, r)

Randomized-Partition(A, l, r)

1. $i \leftarrow \text{Random}(l, r)$
2. exchange $A[l] \leftrightarrow A[i]$
3. return Partition(A, l, r)

Average-Case Splitting

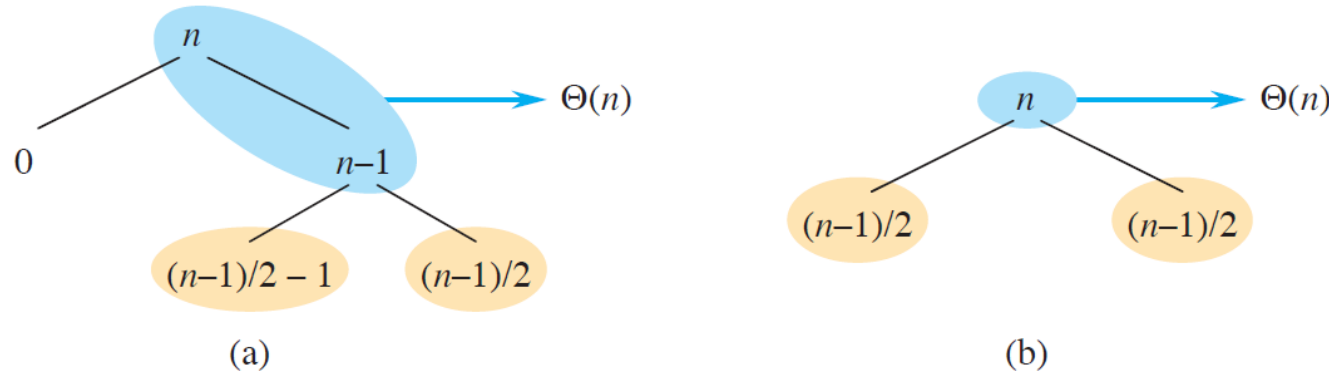


Figure 7.5 (a) Two levels of a recursion tree for quicksort. The partitioning at the root costs n and produces a “bad” split: two subarrays of sizes 0 and $n - 1$. The partitioning of the subarray of size $n - 1$ costs $n - 1$ and produces a “good” split: subarrays of size $(n - 1)/2 - 1$ and $(n - 1)/2$. (b) A single level of a recursion tree that is well balanced. In both parts, the partitioning cost for the subproblems shown with blue shading is $\Theta(n)$. Yet the subproblems remaining to be solved in (a), shown with tan shading, are no larger than the corresponding subproblems remaining to be solved in (b).

The combination of good and bad splits would result in $T(n) = \Theta(n \log n)$, but with slightly larger constant hidden by the O -notation. (Rigorous average-case analysis later)